AI Usage Log

Question 1, Parts 1 and 2

Prepared by	Jeremiah Igbinedion
AI tool used	ChatGPT
Purpose	Document how AI supported the work, the prompts used, and the corrections made before submission.

Declaration

I used ChatGPT as a support tool for analysis, organisation and drafting. I reviewed the output against the supplied requirements, challenged unsupported assumptions, changed the test case selection, refined the priorities and verified that I could explain every item included in the final documents.

1. Question 1, Part 1

Risk Based Test Plan

What I used AI for

I used ChatGPT to help organise my analysis of the digital wallet transfer limits and account tier requirements into a structured risk based test plan. It helped identify unclear requirements, generate clarification questions, group risks by priority and present the test strategy professionally.

Example prompts

EXAMPLE PROMPT 1

> Review the digital wallet transfer limit and account tier requirements. Identify unclear areas, missing business rules, assumptions, dependencies and possible risks. Create clarification questions that should be answered before testing begins.

EXAMPLE PROMPT 2

> Using the supplied requirements and identified risks, create a risk based test plan covering the scope, test approach, risk priorities, functional testing, boundary testing, API testing, concurrency testing, scheduled transfers, KYC tier upgrades, pending transfers, reversals and own account exemptions.

What I reviewed and corrected

I compared the generated questions and plan with the original wording. I removed unsupported statements or treated them as clarification questions instead of confirmed behaviour. I also reorganised the risks according to possible financial loss, customer impact and regulatory exposure.

- Confirmed that the Tier 1, Tier 2 and Tier 3 limits were represented accurately.
- Ensured pending, failed and reversed transfer behaviour was included.
- Added coverage for the midnight reset, scheduled transfers and immediate KYC upgrades.
- Treated transfers to the user’s own linked bank accounts as exempt from the stated limits.
- Raised the authoritative timezone, partial reversals, transfer type aggregation and scheduled failure handling as clarification points because they were not fully specified.
- Prioritised financial loss, limit bypass, duplicate transfers, race conditions and incorrect allowance calculations.

2. Question 1, Part 2

Top 20 Prioritised Test Cases

What I used AI for

I used ChatGPT to convert the supplied requirements and risk based test plan into structured test cases with IDs, priorities, preconditions, steps and expected results.

Example prompts

EXAMPLE PROMPT 1

Create the top 20 prioritised test cases for the digital wallet transfer limits and account tiers feature. Each test case must include an ID, priority, preconditions, steps and expected result. Focus on quality over quantity.

EXAMPLE PROMPT 2

Select 15 test cases that directly validate the seven supplied requirements. Use the remaining five cases for important high risk scenarios, including concurrency, backend limit enforcement, duplicate submissions, insufficient balance and invalid transfer amounts.

ADDITIONAL DIRECTION

We need to choose the cases that directly validate these exact requirements and hints: Tier 1, Tier 2 and Tier 3 limits, midnight reset, pending transfers, failed and reversed transfers, own linked account exemptions, scheduled transfers executing at 8:00 AM, immediate KYC tier upgrades, and rejection messages showing the remaining daily limit.

What I reviewed and corrected

The initial output gave too much attention to unconfirmed behaviours, including idempotency, partial reversals, shared limits across transfer types and cross day ledger handling. I changed the selection so the final set contained 15 cases that directly validated the stated requirements and 5 cases for major technical and financial risks.

3. Corrections Applied to the Test Cases

- Selected 15 cases with direct traceability to the seven supplied requirements.
- Reserved 5 cases for concurrency, backend enforcement, duplicate submissions, insufficient balance and invalid amounts.
- Added direct boundary validation for Tier 1, Tier 2 and Tier 3.
- Separated the validation of pending, failed and reversed transfers.
- Explicitly verified that scheduled transfers execute at 8:00 AM on the scheduled day.
- Verified that a successful KYC upgrade takes effect immediately without requiring a new login.
- Verified that a rejected transfer displays the correct remaining daily allowance.
- Added backend validation so client side controls cannot be used to bypass transfer limits.
- Added concurrency and repeated submission coverage to reduce the risk of exceeding limits or processing a transfer twice.
- Refined expected results to show the effect on wallet balance, available daily limit and transfer status.

Final review and ownership

I manually checked the final test cases against the requirements, refined the priorities and verified the traceability. I can explain why each case was selected, the data required, the risk it covers and the expected result. The submitted documents are not unreviewed AI output.

Summary	AI supported analysis and drafting. Final scope, prioritisation, corrections and acceptance of the output were completed through my review.

Part Two, API Test Automation

TypeScript, Playwright, Zod and Docker Compose

Candidate	Jeremiah Igbinedion
AI tool used	OpenAI Codex/ChatGPT
Purpose	Document how AI supported manual API investigation and the conversion of that work into a maintainable automated test suite.

How AI was used

I first performed manual API testing against the pinned local Restful Booker service and inspected responses using curl and controlled exploratory requests. AI then helped convert the verified manual scenarios into structured automation, while also supporting framework design, investigation, documentation and verification.

- Broke the implementation into incremental phases and clear commit boundaries.
- Designed the Playwright API framework, typed client, fixtures, factories, schemas and cleanup helpers.
- Converted verified manual authentication, CRUD, negative, boundary and filtering checks into deterministic automated tests.
- Compared expected and actual API behaviour and helped draft defect reports for confirmed findings.
- Ran linting, strict type checks, test suites, repeatability checks and Git verification.
- Did not commit, push, merge or change branches without my explicit approval.

1. Work Completed With AI Assistance

Environment and framework

- Configured a reproducible Docker Compose service using a pinned Restful Booker image digest, localhost-only access, environment-based port configuration and a /ping health check.
- Added linux/amd64 configuration for Apple Silicon compatibility and documented start, status and stop commands.
- Built reusable Playwright and TypeScript components instead of a flat API script.
- Added environment configuration, RestfulBookerClient, fixtures, booking factories, cleanup tracking, Zod schemas, HTML and JUnit reports, ESLint and strict TypeScript checks.

Automated functional coverage

- Authentication and authorization: valid and invalid credentials, missing and malformed bodies, valid, missing, invalid and stale tokens, plus PUT, PATCH and DELETE protection.
- CRUD and schema validation: create, retrieve, full update, partial update, delete, post-delete 404 verification, persistence checks and a complete lifecycle test.
- Negative and boundary testing: missing fields, blank names, invalid price values and types, Boolean coercion, invalid dates, malformed bodies, long values, unknown fields and injection-style strings.
- Filtering: firstname, lastname, combined names, case sensitivity, check-in, checkout, date ranges, exact boundaries, invalid dates, no-match results, special characters, URL encoding and unsupported pagination parameters.
- Filtering tests retrieve each returned booking ID and validate the booking data instead of treating HTTP 200 alone as proof of correctness.

Defect investigation

AI helped structure the confirmed findings as BUG-API-001 through BUG-API-016, including reproduction steps, expected and actual behaviour, severity, impact, repeatability evidence and links to automated tests. I reviewed the classifications and challenged findings that were not adequately supported.

2. Representative Prompts

PROMPT 1, FRAMEWORK SETUP

Create a reproducible local Restful Booker environment and a minimal TypeScript/Playwright API framework. Use fixtures, helpers, configuration, response schemas, HTML and JUnit reports, and a health test. Keep the target localhost-only. Do not commit or push without my approval.

PROMPT 2, AUTHENTICATION COVERAGE

Implement authentication and authorization coverage for valid credentials, invalid username and password, missing and empty credentials, malformed bodies, valid tokens, missing tokens, invalid tokens, and stale-token behavior. Every protected-endpoint test must create and clean up its own booking.

PROMPT 3, CRUD INVESTIGATION

Investigate POST, PUT, PATCH, and DELETE against a local Restful Booker instance in a separate temporary folder before adding tests to the repository. Use only records created during the investigation, verify persistence, clean everything up, and report confirmed bugs separately from product questions.

PROMPT 4, NEGATIVE AND FILTERING COVERAGE

Test wrong types, malformed bodies, price boundaries, invalid dates, injection-style strings, name filters, date-range filters, case sensitivity, pagination behavior, and special-character encoding. Verify filtering correctness by retrieving the returned booking IDs, not just by checking for HTTP 200.

3. Human Corrections and Decisions

Configuration and Git safety

- Required explicit approval before every commit or push and reviewed each proposed scope and commit message.
- Corrected hard-coded admin credentials in the fixture so username and password came from environment configuration and could be overridden in CI or other environments.

Evidence-based defect classification

- Did not classify duplicate-date bookings as a confirmed defect because the API exposes no room, property, capacity or resource identifier. Added a characterization test and documented the overbooking risk as a product question.
- Kept unauthenticated GET /booking access as an observation because the documented test API may intentionally allow public reads.
- Used Playwright expected failures only for confirmed defects. Product questions and limitations were not marked with test.fail.
- Limited token-expiry claims to the available evidence: fabricated stale-token rejection, restart invalidation, no deterministic natural expiry and no documented expiration mechanism.
- Retained case-sensitive filtering as a usability or product question because the documentation does not specify case-insensitive matching.

Wire-level request correction

Exploratory JavaScript fetch and Playwright initially produced different results for empty bodies. I required further investigation with curl --data-binary ''. The client was corrected to send raw bodies using a Buffer, ensuring a genuinely zero-byte request. The resulting POST, PUT and PATCH automation then matched the observed wire behaviour.

4. Confirmed Findings Supported by Automation

- BUG-API-001, Invalid credentials return HTTP 200.
- BUG-API-002, Tokens have no expiration or revocation mechanism.
- BUG-API-003, GET /ping returns HTTP 201.
- BUG-API-004, Missing or empty credentials receive generic authentication errors.
- BUG-API-005, Negative booking prices are accepted.
- BUG-API-006, Blank and whitespace-only customer names are accepted.
- BUG-API-007, Missing mandatory booking fields produce HTTP 500.
- BUG-API-008, Non-boolean deposit values are accepted and coerced.
- BUG-API-009, Invalid date ordering and zero-night stays are accepted.
- BUG-API-010, Operations on nonexistent bookings return HTTP 405.
- BUG-API-011, Booking field types are inconsistently validated.
- BUG-API-012, Price coercion and truncation cause data corruption.
- BUG-API-013, Impossible calendar dates are silently normalized.
- BUG-API-014, Empty and array POST bodies cause HTTP 500.
- BUG-API-015, Check-in filtering excludes the documented lower boundary.
- BUG-API-016, Invalid filtering dates produce HTTP 500.

Final review and ownership

I manually executed and inspected API requests, reviewed the AI-assisted automation, corrected unsupported assumptions and implementation details, and verified the final suite through linting, type checks, repeated test runs and result inspection. I can explain the framework, each test, each confirmed defect and the corrections made. The submitted work is not unreviewed AI output.

Part 5, CI/CD

AI usage supplement, appended without changing the previous nine pages

AI tool	OpenAI Codex / ChatGPT
Purpose	Support CI/CD planning, implementation, review, debugging, documentation and verification.
Final status	Pipeline completed and GitHub Pages dashboard live

How AI was used

- Reviewed the Part 5 requirements and divided implementation into progressive milestones.
- Designed the lint and type-check gate, four-shard matrix and pinned Docker service.
- Prepared Playwright for test-level sharding and designed known-bug expected-failure handling.
- Implemented artifact uploads, blob-report merging, combined HTML and JUnit reports, and the Pages dashboard.
- Reviewed secret handling, workflow YAML, local verification results and real Actions runs.
- Helped identify and correct security, sharding and reporting issues.

Representative prompts

PROMPT 1, PIPELINE DESIGN

Create a plan for Part 5 that runs lint before the complete Part 2 Playwright API suite, starts Restful Booker as a GitHub Actions service, runs jobs in parallel where possible, uploads reports and failure artifacts, handles expected-failure tests without keeping CI red, and includes nightly execution and test sharding.

PROMPT 2, REPORTING AND PAGES

Replace the separate shard HTML reports with one combined Playwright report, improve the final HTML presentation, add a professional dashboard with recent workflow history, and prepare it for GitHub Pages.

Human direction

- Required credentials to come from environment variables and GitHub Secrets.
- Required explicit approval before commits or pushes.
- Selected one merged report instead of four shard reports.
- Selected successful main runs only for Pages deployment.
- Selected a professional light dashboard with recent run history and generic Restful Booker identity.

Corrections made to AI-assisted output

Shard distribution

Changed file-level sharding from 27, 53, 0 and 21 tests to test-level distribution of 26, 25, 25 and 25.

Credential security

Removed duplicated hardcoded username and password values and required configuration through environment variables and secrets.

Report usability

Replaced four independent shard reports with merged blob reporting and one combined HTML and JUnit result.

Pages privacy

Excluded error contexts, JUnit XML, ZIP archives, traces, screenshots and other diagnostics from the public bundle.

Deployment safety

Restricted deployment to successful main runs so branches, pull requests and failed runs cannot overwrite the public green report.

Human verification and ownership

- Reviewed workflow triggers, dependencies, service configuration, credentials, sharding, expected failures and artifact handling.
- Ran and checked linting, TypeScript, Playwright, shard distribution and report merging.
- Reviewed feature and develop Actions runs, artifact inventories, JUnit totals and dashboard layouts.
- Confirmed that the GitHub Pages dashboard is live at https://dionjerry.github.io/quality-engineering-project/.
- Approved the implementation decisions and can explain or modify the pipeline during the live defence.

Final responsibility statement

AI supported planning, implementation, debugging, documentation and verification. I selected the architecture and deployment policies, corrected the credential, sharding, reporting and privacy issues, supplied repository secrets, approved Git operations and verified the resulting behaviour. The submitted work is reviewed and understood, not unexamined AI output.